AWS 3-Tier Highly Available Web Application (Step-by-Step Lab Guide)

Project Overview

Explain in simple words:

What you built:

- A scalable web application hosted on AWS
- Load-balanced and auto-scaled across multiple AZs

Why:

- Handle high traffic (like sales events)
- Ensure high availability and fault tolerance

AWS Services Used

List and explain briefly:

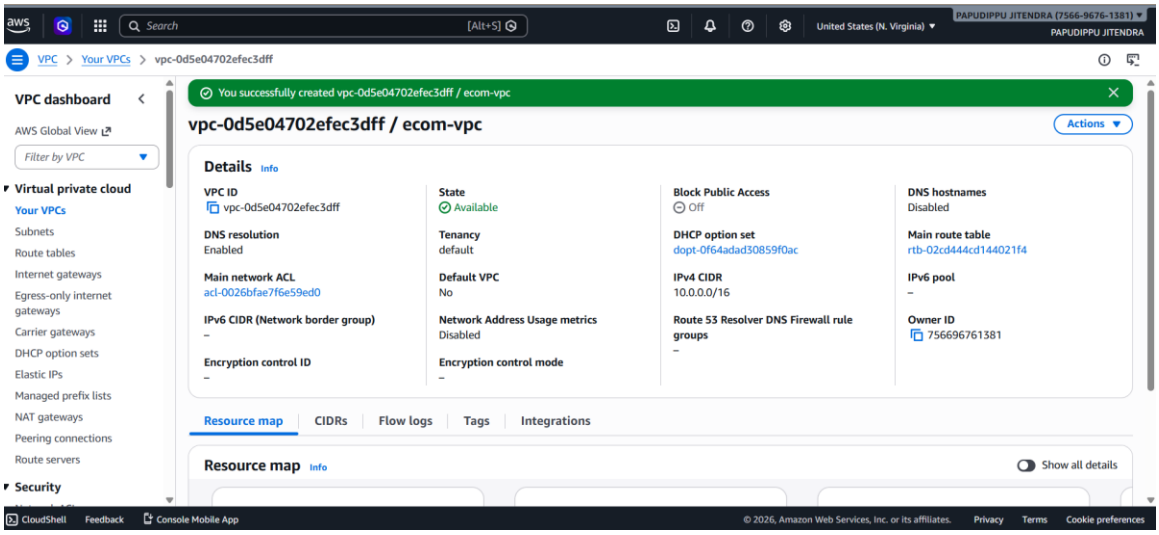
- Amazon EC2 → Hosting application
- Auto Scaling Group → Auto scaling
- Application Load Balancer → Traffic distribution

Objectives

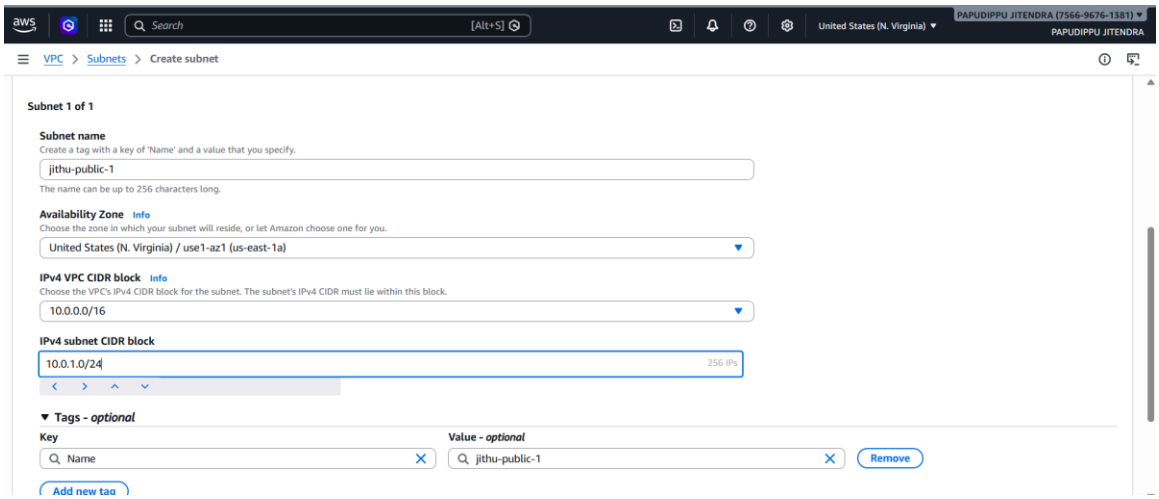
- Build a highly available 3-tier web application on AWS
- Deploy resources across multiple Availability Zones
- Configure Application Load Balancer for traffic distribution
- Implement Auto Scaling Group for automatic scaling
- Use Amazon EC2 instances to host the application
- Secure the application using HTTPS via AWS Certificate Manager
- Store static files in Amazon S3
- Use Amazon EBS for persistent storage

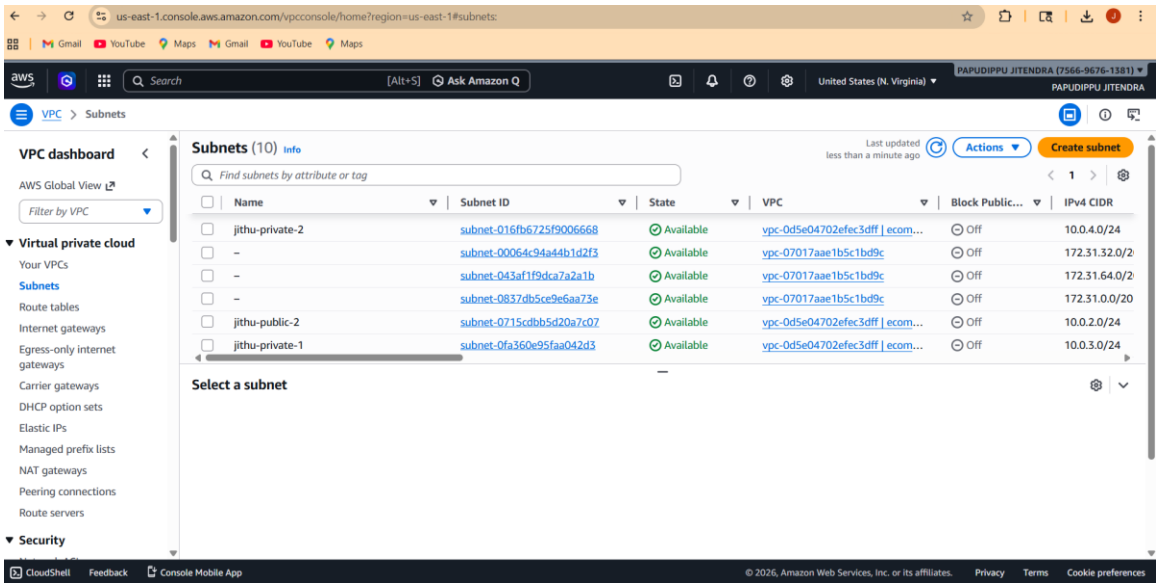
- Create a reusable Golden AMI for faster deployment
- Ensure fault tolerance and high availability
- Test load balancing and auto scaling behavior

PART 1 — VPC SETUP (GUI STEPS)
Go to: AWS Console → VPC → Create VPC
Name: ecom-vpc
CIDR: 10.0.0.0/16
Enable DNS hostnames

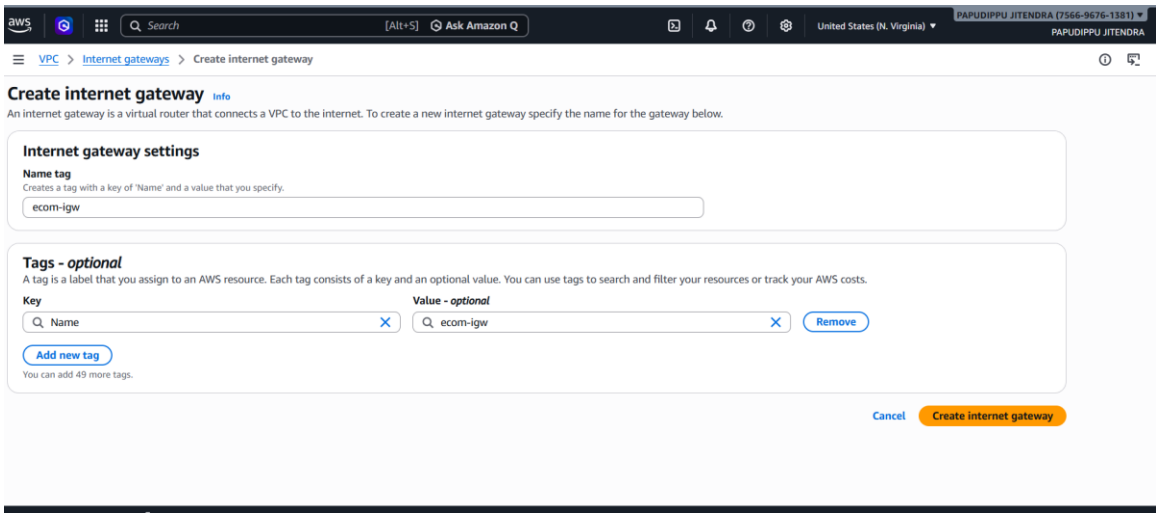


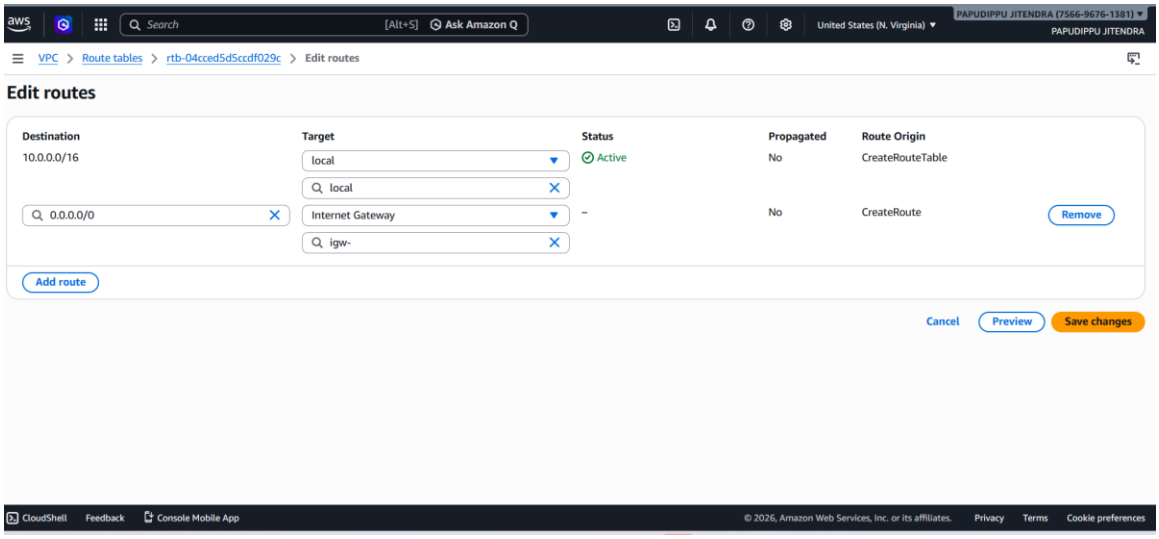
Create Subnets:
Public:
- public-1 → AZ-a → 10.0.1.0/24
- public-2 → AZ-b → 10.0.2.0/24
Private:
- private-1 → AZ-a → 10.0.3.0/24
- private-2 → AZ-b → 10.0.4.0/24



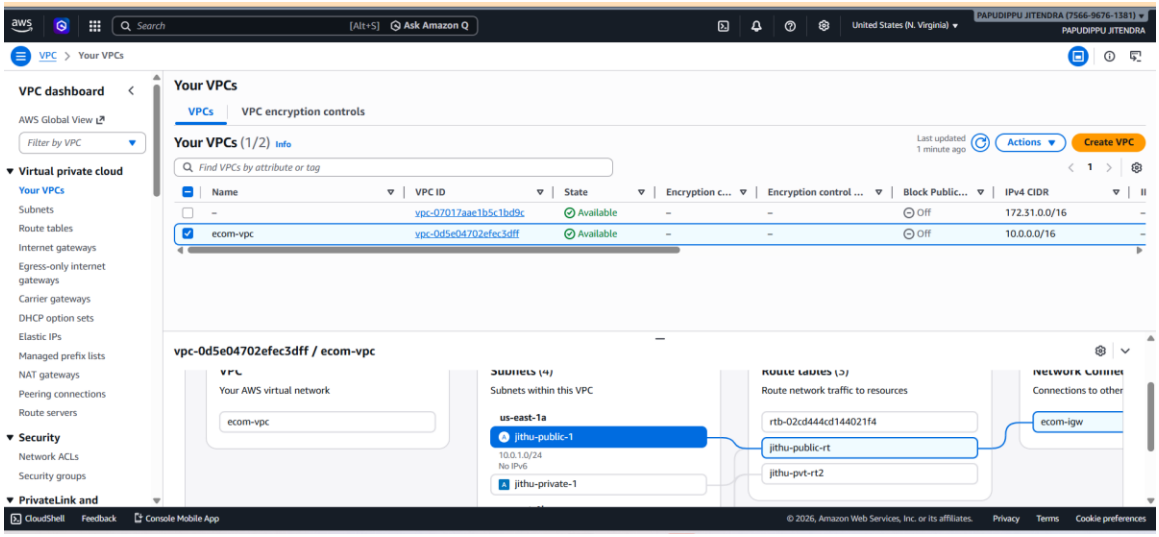


Internet Gateway:
Create → ecom-igw → Attach to VPC

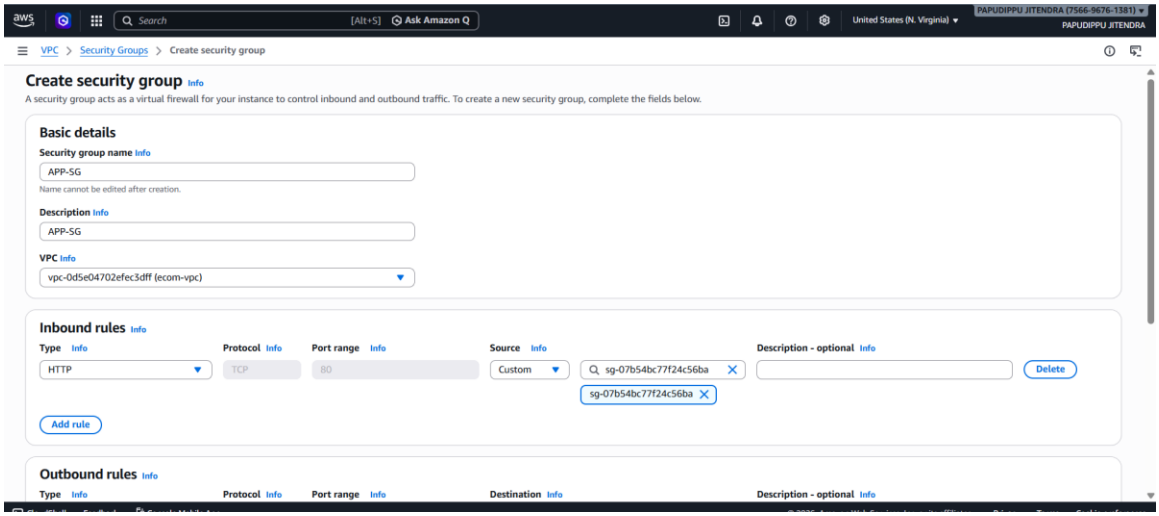


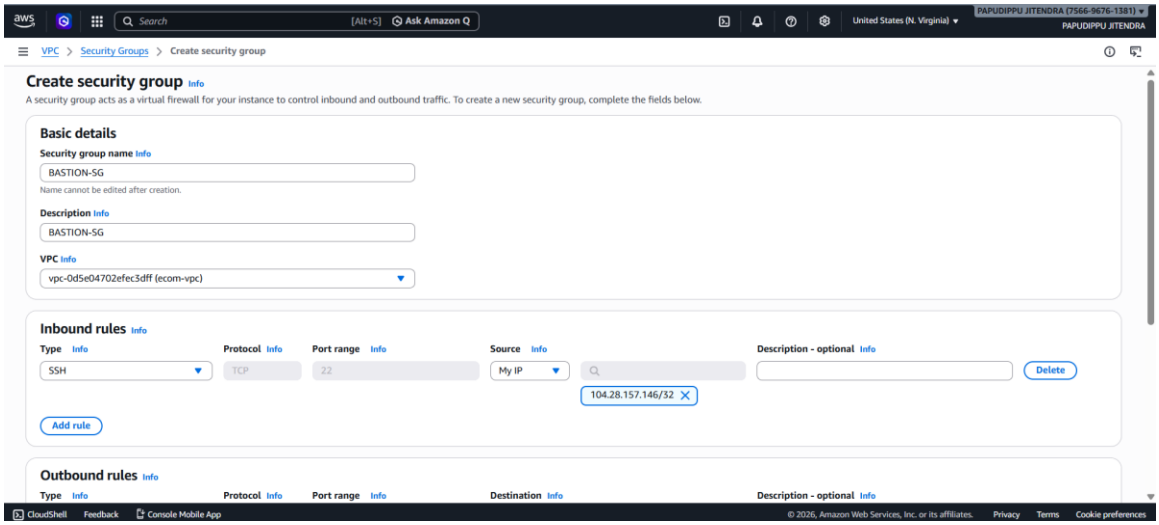


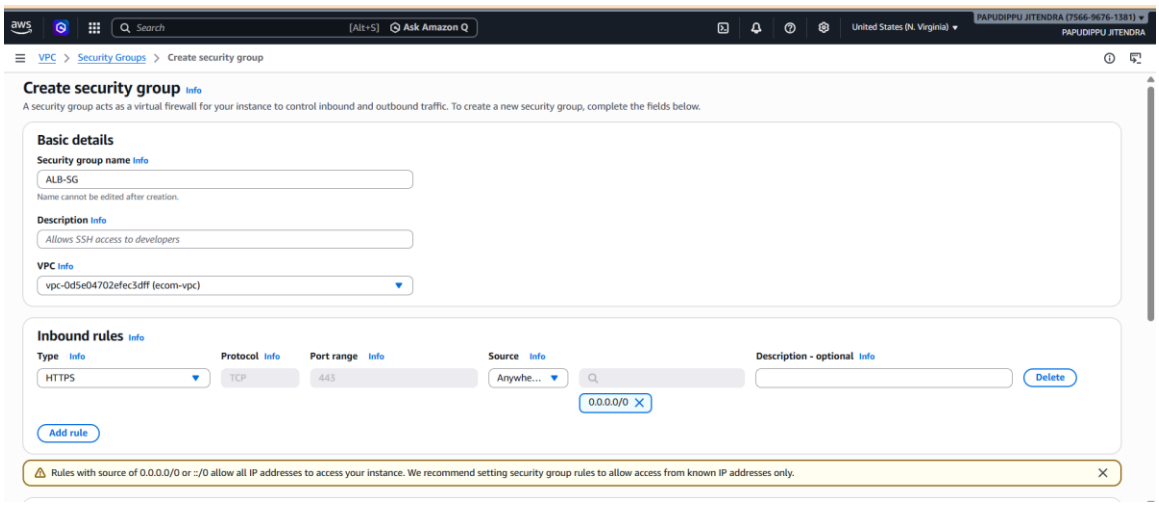
Route Table:
Create public-rt
Add route: 0.0.0.0/0 → IGW
Associate public subnets



PART 2 — SECURITY GROUPS
ALB-SG: HTTPS (443) → Anywhere
APP-SG: HTTP (80) from ALB-SG, SSH (22) from Bastion-SG
BASTION-SG: SSH (22) from My IP







PART 3 — BASTION HOST
Launch EC2:
Name: bastion
AMI: Ubuntu
Instance: t2.micro
Subnet: public-1
Enable public IP

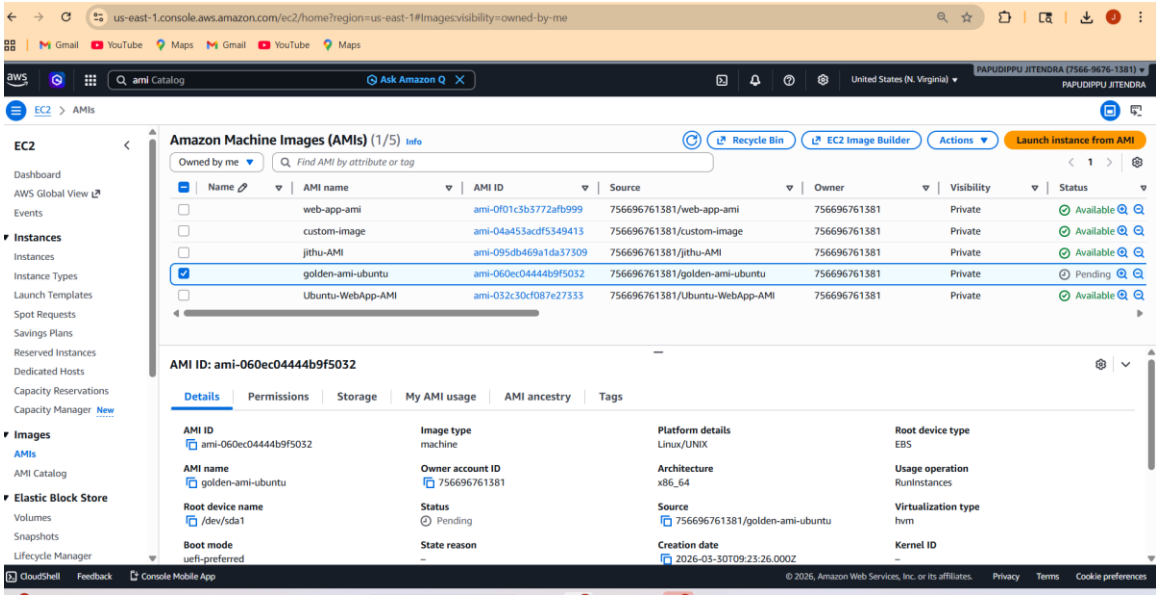
Attach Elastic IP

PART 4 — GOLDEN AMI (APT)

Launch Ubuntu EC2 → SSH

Commands:
sudo apt update -y
sudo apt install apache2 -y
echo "Hello from Golden AMI $(hostname)" | sudo tee /var/www/html/index.html
sudo systemctl enable apache2
sudo systemctl start apache2

Create AMI → Name: golden-ami-ubuntu

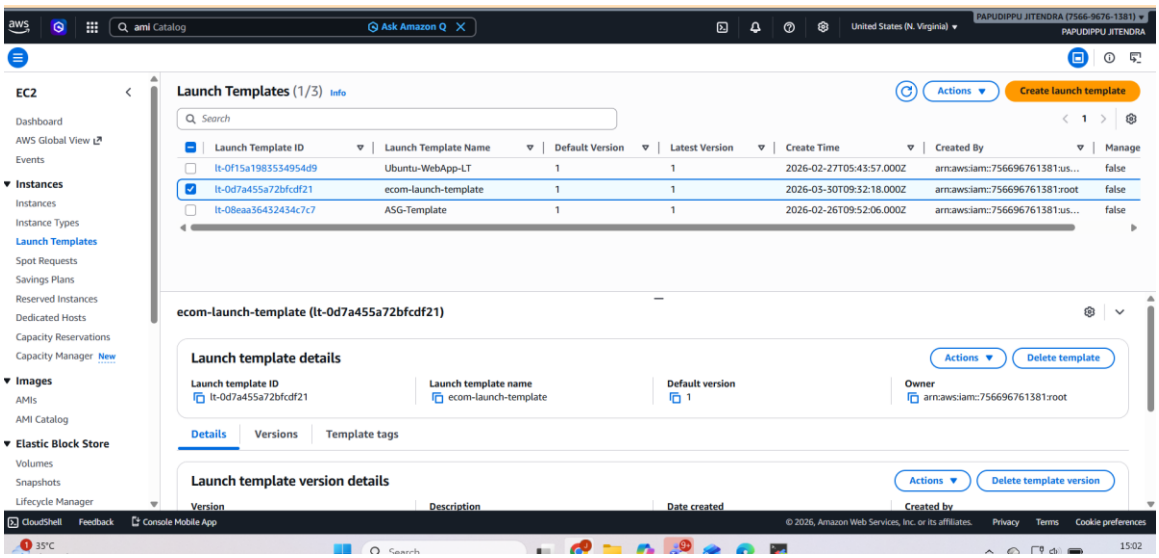


Hello from Golden AMI ip-10-0-1-111

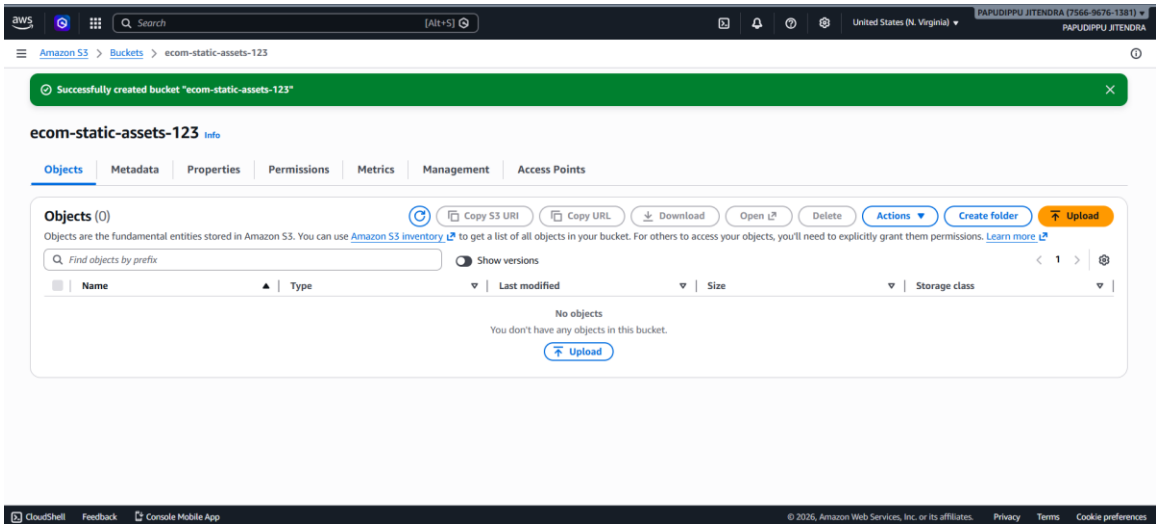
PART 5 — LAUNCH TEMPLATE
Use Golden AMI
Instance: t3.micro
Security Group: APP-SG

Storage:
20GB gp3 (Encrypted)

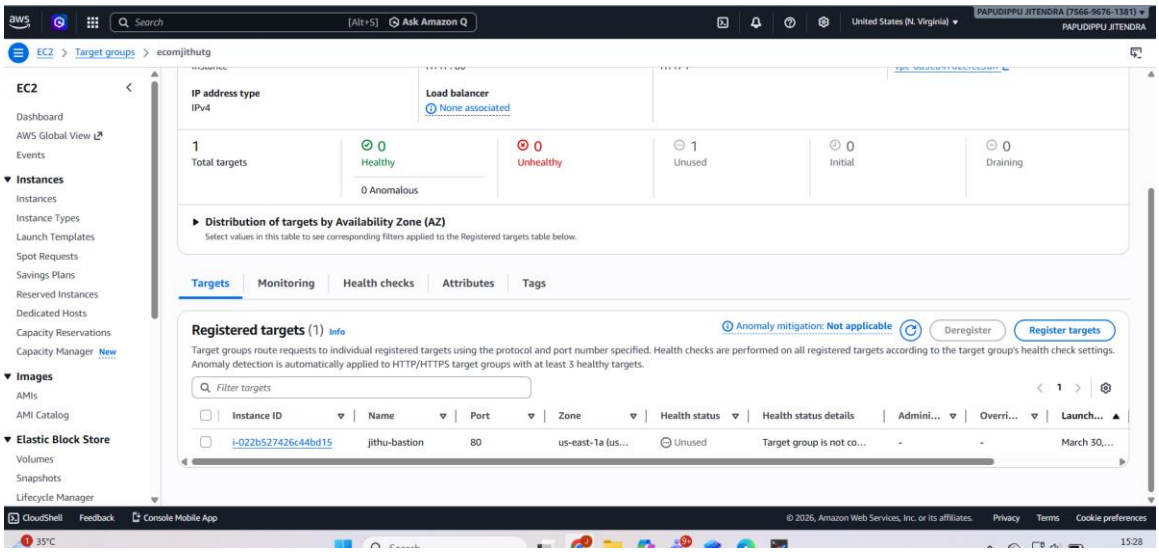
User Data:
#!/bin/bash
apt update -y
apt install apache2 -y
echo "Web Server Running on $(hostname)" > /var/www/html/index.html
systemctl enable apache2
systemctl start apache2



PART 6 — S3 BUCKET
Create bucket: ecom-static-assets-123
Enable versioning
Upload images & CSS

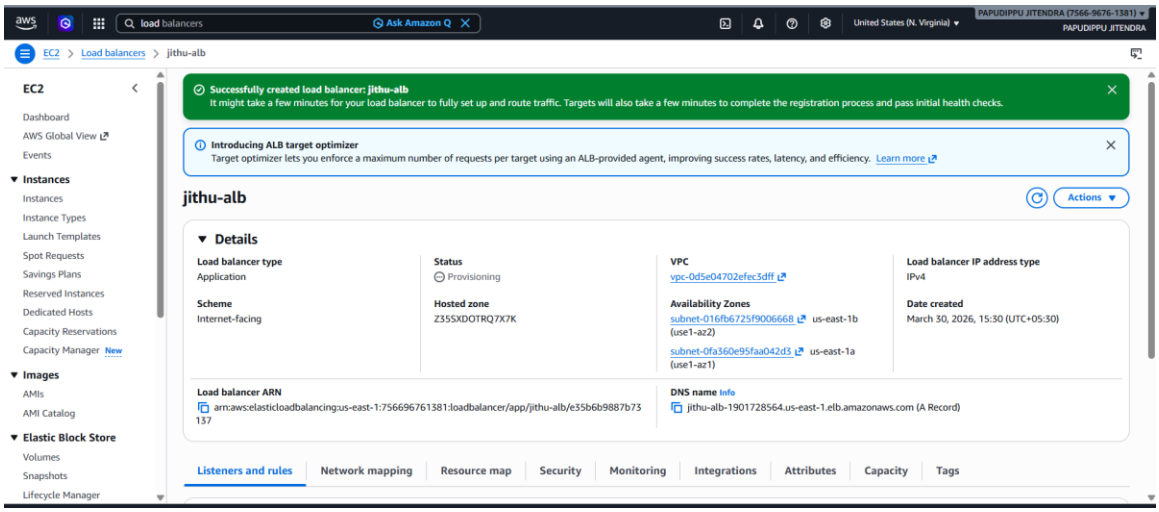


PART 7 — TARGET GROUP
Protocol: HTTP (80)
Health check: /



PART 8 — APPLICATION LOAD BALANCER
Name: ecom-alb
Internet-facing
Subnets: public-1, public-2
Security Group: ALB-SG
Listener: HTTP(80)

Enable:
Cross-zone load balancing
Sticky sessions (1 hour)

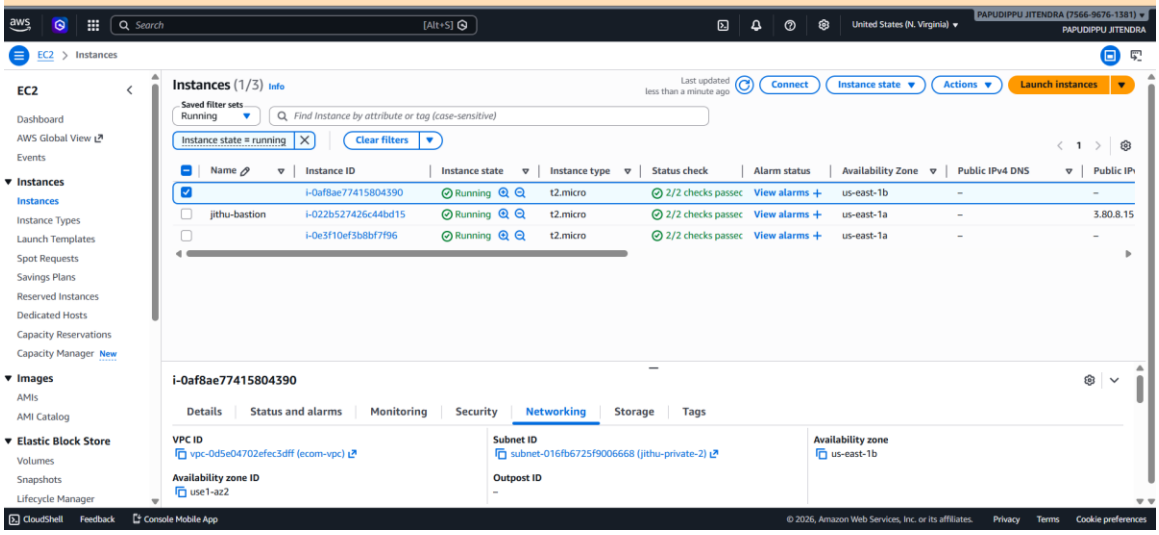


PART 9 — AUTO SCALING GROUP
Use Launch Template
Subnets: private-1, private-2
Attach Target Group

Capacity:
Min: 2 | Desired: 2 | Max: 4

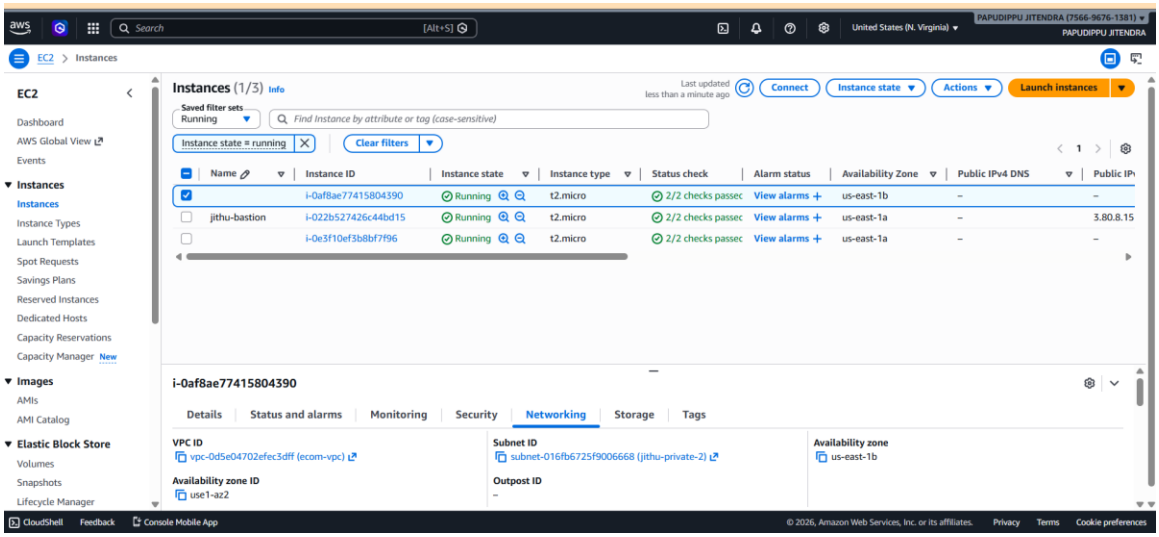
Scaling:

CPU 50%
Cooldown: 300 sec



PART 10 — TESTING
Test ALB DNS
Refresh → Load balancing
Stickiness test
Run:
sudo apt install stress -y
stress --cpu 2

Stop instance → test HA



QUICK DEBUG
ALB 503 → Check health check & Apache
Connection issue → Check SG
Scaling issue → Check CPU

Web Server Running on ip-10-0-4-197

FINAL OUTPUT
EC2 + ASG
ALB
S3
EBS
Multi-AZ Architecture

Challenges Faced

Example:

- Target group unhealthy
- Security group misconfiguration
- SSH issues via bastion

Conclusion

This project successfully implemented a highly available and scalable 3-tier web application on AWS using services like Amazon EC2, Application Load Balancer, and Auto Scaling Group. The system ensures efficient traffic distribution, automatic scaling, and fault tolerance across multiple Availability Zones. Overall, it demonstrates a reliable and production-ready cloud architecture capable of handling dynamic workloads.